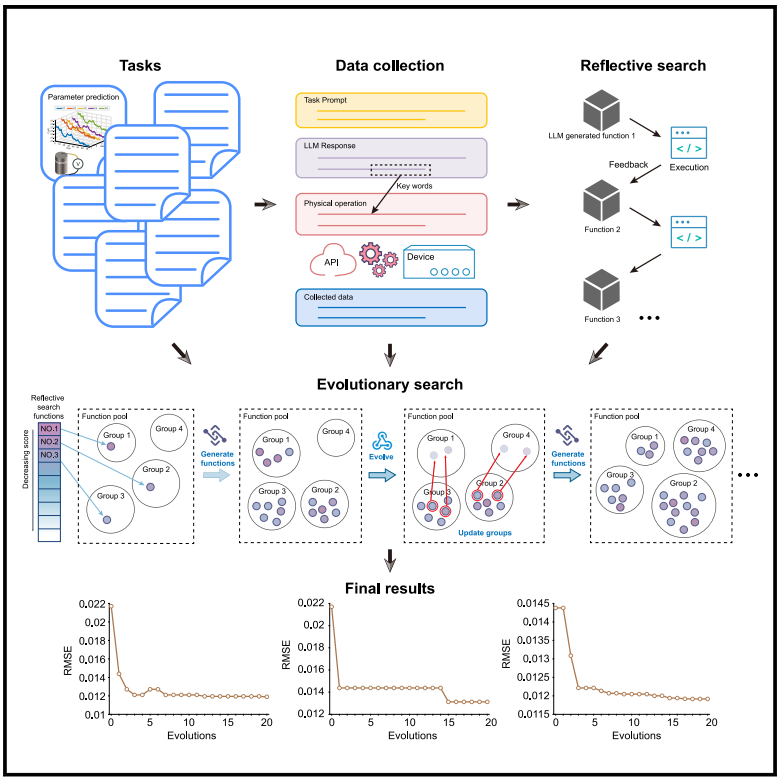
Article

Patterns

RePower: An LLM-driven autonomous platform for power system data-guided research

Graphical abstract



Highlights

- An automated platform integrates data acquisition, algorithm design, and evolution
- Results improve continuously using a two-stage reflection-evolution search framework
- The framework is validated with six data-driven tasks in the power system
- Our evolutionary strategy outperforms other frameworks in comparative tests

Authors

Yu-Xiao Liu, Mengshuo Jia, Yong-Xin Zhang, Jianxiao Wang, Guannan He, Shao-Long Zhong, Zhi-Min Dang

Correspondence

jia@eeh.ee.ethz.ch (M.J.), dangzm@tsinghua.edu.cn (Z.-M.D.)

In brief

The authors present an automatic scientific research platform called RePower, which utilizes a two-stage reflection-evolution strategy for long-context thinking and reasoning to complete data-driven tasks of power systems. This framework can better assist large language models (LLMs) in accomplishing complex research tasks, including physical and virtual ones, further promoting the application of generative artificial intelligence in the field of scientific research.

Liu et al., 2025, Patterns 6, 101211
April 11, 2025 © 2025 The Author(s). Published by Elsevier Inc.
https://doi.org/10.1016/j.patter.2025.101211

Patterns

CellPress
OPEN ACCESS

Article

RePower: An LLM-driven autonomous platform for power system data-guided research

Yu-Xiao Liu,[1] Mengshuo Jia,[2,*] Yong-Xin Zhang,[1] Jianxiao Wang,[3] Guannan He,[4] Shao-Long Zhong,[1] and Zhi-Min Dang[1,5,*]

[1]State Key Laboratory of Power System Operation and Contral, Department of Electrical Engineering, Tsinghua University, Beijing 100084, P.R. China

[2]Power Systems Laboratory, Department of Information Technology and Electrical Engineering, ETH Zürich, 8092 Zürich, Switzerland

[3]National Engineering Laboratory for Big Data Analysis and Applications, Peking University, Beijing 100871, P.R. China

[4]Department of Industrial Engineering and Management, College of Engineering, Peking University, Beijing 100871, P.R. China

[5]Lead contact

*Correspondence: jia@eeh.ee.ethz.ch (M.J.), dangzm@tsinghua.edu.cn (Z.-M.D.)

https://doi.org/10.1016/j.patter.2025.101211

THE BIGGER PICTURE Artificial intelligence (AI) has transformed numerous industries in the 21st century, significantly advancing the acquisition, processing, and development of algorithms in scientific research. The rapid growth of the AI industry has led to the emergence of generative artificial intelligence, such as large language models (LLMs) like GPT, which have demonstrated exceptional capabilities in handling language tasks across various fields. Due to their advanced understanding and reasoning abilities, LLMs are increasingly being applied in scientific research and have the potential to drive new discoveries and enhance research efficiency. In existing studies, scientists have already utilized LLMs to automate research in multiple domains, such as chemistry, mathematics, and materials science. If these frameworks can be successfully transferred to the field of power systems or expanded further in other areas, then the full potential of generative AI could be harnessed to support researchers in conducting efficient scientific work.

SUMMARY

Large language models (LLMs) have shown strong capabilities across disciplines such as chemistry, mathematics, and medicine, yet their application in power system research remains limited, and most studies still focus on supporting specific tasks under human supervision. Here, we introduce Revive Power Systems (RePower), an autonomous LLM-driven research platform that uses a reflection-evolution strategy to independently conduct complex research in power systems. RePower assists researchers by controlling devices, acquiring data, designing methods, and evolving algorithms to address problems that are difficult to solve but easy to evaluate. Validated on three critical data-driven tasks in power systems—parameter prediction, power optimization, and state estimation—RePower outperformed traditional methods. Consistent performance improvements were observed across multiple tasks, with an average error reduction of 29.07%. For example, in the power optimization task, the error decreased from 0.00137 to 0.000825, a reduction of 39.78%. This framework facilitates autonomous discoveries, promoting innovation in power systems research.

INTRODUCTION

Power systems are among the most critical and complex infrastructures in modern society, consisting of interconnected components that manage the generation, transmission, distribution, monitoring, and regulation of electrical energy.[1] In power systems research, tasks are often structured through modular algorithms or solution frameworks. Exploring the optimal modular algorithm is not a conventional optimization problem, prompting researchers to turn to artificial intelligence (AI) as a promising approach to tackle this challenge.[2–4]

With the emergence and development of generative AI and large language models (LLMs),[5–7] scientists have begun to build intelligent agents or automated research assistants to assist people in completing complex tasks.[8–11] In the field of chemical research, scholars have proposed and developed intelligent agents such as Coscientist[12] and ChemCrow.[13] These types of agents are capable of performing chemical synthesis planning

This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

and corresponding physical operations, effectively promoting the development and advancement of autonomous scientific research using LLM. In the field of mathematical computation, scholars have proposed Funsearch,[14] an intelligent agent based on feedback-driven autonomous search heuristics, which is capable of finding superior solutions to combinatorial mathematics problems compared to existing ones. However, these agents cannot be directly applied to power systems research. Agents in chemistry are designed for short-text, short-memory-chain reasoning processes within their domain. However, the tasks of power systems often require complex reasoning processes and long chains of thinking and memory. If the design of the prompt and the reasoning process of the LLM is not properly considered, then there is a high likelihood of errors occurring, which could lead to task interruption. While mathematical computation agents can evolve algorithms, their capabilities are confined to combinatorics and do not extend to general problems such as prediction in power systems.

Despite the progress in other fields, the use of LLMs as automated research assistants in power systems remains underexplored. Some scholars have designed agent frameworks for simple tasks in power systems, such as document retrieval or knowledge queries.[15,16] Some of these frameworks include direct feedback mechanisms, enabling multi-step reasoning.[15] However, compared to the field of chemistry, these agents do not integrate all functionalities into a single framework; instead, they design specific frameworks to achieve a particular function, such as document retrieval. Additionally, most of these tasks involve simple question-and-answer interactions with short contexts, and there is no design for feedback on long memory chains. Other scholars have incorporated LLMs as a component in their algorithmic constructions.[17,18] Arnautov and Akimov only used LLM for the parameter design of a three-node power system, including direct feedback on whether the parameters are correct,[17] while Yan and Xu employed a feedback mechanism that evaluates performance based on output results to optimize system performance in real time.[18] However, the generative capabilities of LLMs are not fully utilized; instead, they are used as traditional tools. Furthermore, none of the existing studies involve the control of physical devices,[19,20] which represents a significant gap compared to advancements in the field of chemistry. Consequently, the full potential of LLMs as autonomous research designers in this field has yet to be realized.

Designing an LLM-based automated research assistant for power systems poses distinct challenges due to the complex and often prolonged reasoning required in these tasks.[19,20] Existing LLM research assistants, such as FunSearch in mathematics,[14] are capable of solving predefined tasks with clear structures but struggle with tasks demanding extensive reasoning. Although power systems research tasks can sometimes be reframed as data-driven, end-to-end processes requiring less reasoning,[21–23] they still involve intricate designs. These include challenges such as feature engineering, algorithm development, technique modification, and hyperparameter tuning. Additionally, power systems research often requires interaction with physical equipment, a capability beyond the current scope of LLM-based assistants like FunSearch.

To address the challenges in power systems research and unlock the full potential of LLMs, we introduce RePower (Revive Power Systems), an auto-evolving research platform designed to autonomously create and refine solution algorithms. RePower operates in two distinct stages. Stage 1 employs self-reflection mechanisms[24] to sequentially explore and design various methods, while stage 2 refines these solutions through function pools and evolutionary algorithms, enabling substantial improvements through both broad (horizontal, prioritizing trying more different methods) and in-depth (vertical, prioritizing improving and adjusting existing methods) explorations. Unlike existing tools like FunSearch, which evolve only parts of solutions predefined by researchers, RePower constructs comprehensive solutions from the ground up in its first stage. Moreover, RePower's integration with physical equipment allows it to control devices and acquire data, enhancing its capability to automate the research process, including obtaining experimental data from measurement equipment, exploring methods using existing data, evolving algorithms for current methods, and providing the code and errors during the whole evolutionary process. By leveraging multiple feedback loops, RePower supports human researchers throughout the entire research cycle, from initial algorithm design to final result improvement.

To evaluate RePower's effectiveness, we validated its performance on three key tasks in power systems: parameter prediction, power optimization, and state estimation. These tasks represent typical data-driven, supervised learning challenges commonly encountered in the field. Additional evaluations of more complex tasks and comparisons with other LLM-based evolution frameworks are detailed in the supplemental information, demonstrating RePower's adaptability and superiority to complex scenarios. The results demonstrate RePower's strong potential as a research assistant in power systems. RePower outperforms existing methods in parameter prediction and shows continuous improvement on tasks without established benchmarks. Compared to the single-stage FunSearch method, RePower's dual-stage approach significantly enhances overall performance. Notably, for tasks with unknown answers, RePower's ability to autonomously refine results mirrors human-like exploration, underscoring its capability to navigate and excel in uncharted research domains.

RESULTS

Structure of RePower

RePower is a reflection-evolution system designed to automate task completion by integrating multiple components, as illustrated in Figure 1. As depicted in Figure 1A, RePower begins the research process by collecting data and retrieving information directly from physical devices, seamlessly integrating the entire scientific research workflow. The system's final outputs can also be transmitted back to these devices for further testing and validation. For a detailed description of each component's configuration and design, please refer to the methods.

Preparation

To enable RePower to perform a specific research task, three key elements must be prepared in addition to the data: (1) base knowledge, (2) task descriptions, and (3) evaluation criteria. Base knowledge provides task-independent implementation steps and outlines specific requirements necessary for

CellPress
OPEN ACCESS

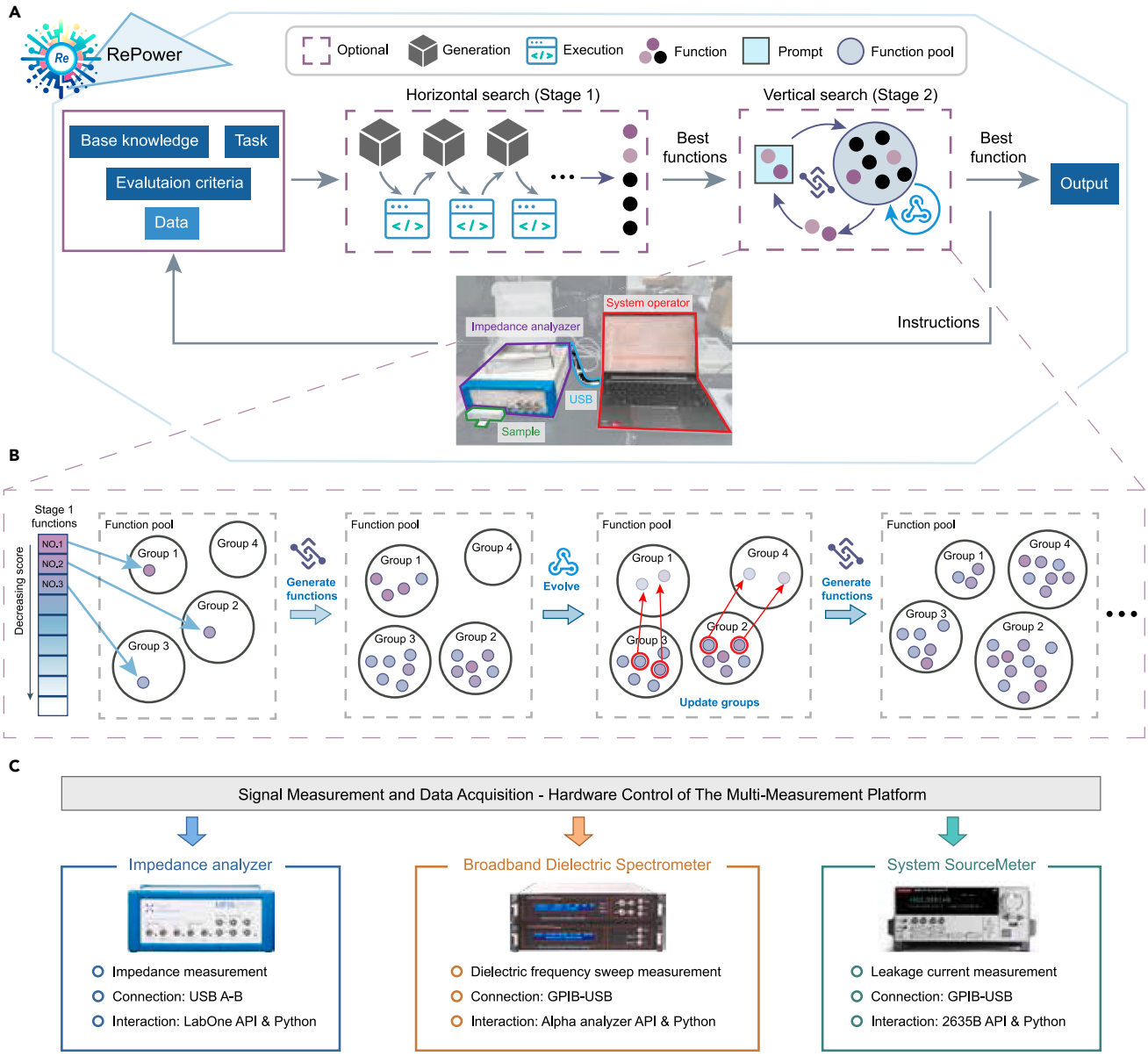


Figure 1. Structure of RePower

(A) Workflow of RePower. The process begins with the preparation of base knowledge and evaluation criteria. If iterative evolution is selected, then the system undergoes a two-stage search process rather than directly outputting results. In stage 1, the LLM explores different methods using a self-reflection mechanism.[24] The best methods or functions identified in stage 1 are then sent to stage 2 for further refinement if needed. Stage 2 uses extensive generation and evolution to optimize the function, producing the final output. In each iteration, the LLM generates a new function based on feedback.

(B) Evolution process of the function pool. Functions generated in stage 1 are ranked by performance, with top-performing functions entering function groups within the function pool. Each evolution round consists of two steps: sample generation and pool evolution. Samples are generated by merging the best and worst functions from a randomly selected group and added to a group with similar performance. Once the function pool reaches its maximum sample size, two functions are randomly selected from the highest-performing groups to replace functions in other groups, continuing the evolution process until the maximum number of rounds is reached.

(C) Measurement devices for data acquisition. Devices such as the impedance analyzer, broadband dielectric spectrometer, and system sourcemeter are used for data collection. The system can be extended to other API-based instruments, including phasor measurement units in power grids, allowing flexible integration of additional measurement tools.

conducting the research. Task descriptions detail the scientific problem to be addressed, including the task's background, expected inputs and outputs, and the data source. Evaluation criteria are used to assess the rationality and accuracy of the LLM's output, including the execution of code steps and the correctness of the results obtained.

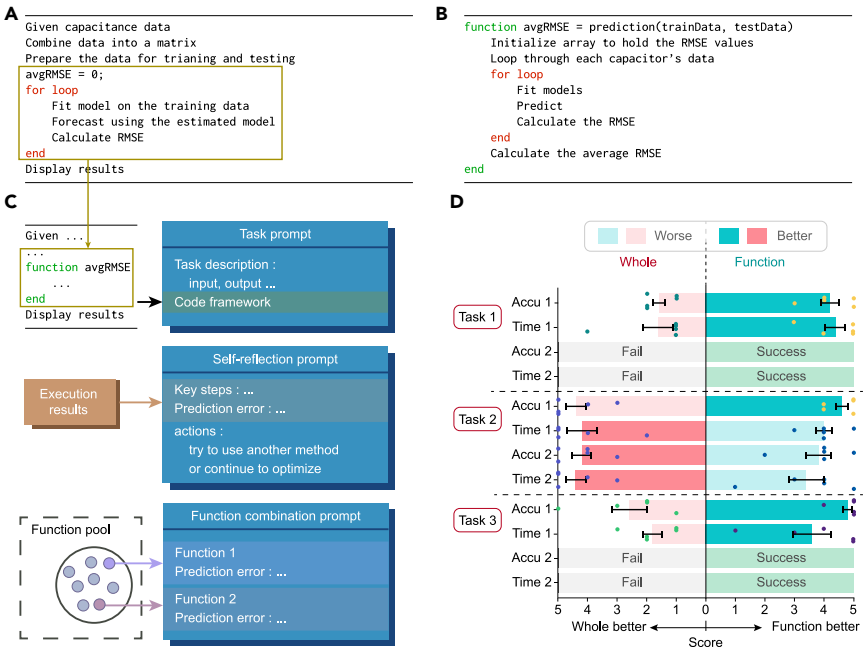


Figure 2. Characterization of the prompt and generation

(A) Whole-code generation. This example demonstrates MATLAB code generation, though the approach is not limited to this type of code.

(B) Function-only generations. In this approach, the original data and error calculation functions are defined externally, focusing solely on the function generation.

(C) Prompt construction. Task prompt is the combination of task description and the code framework, which can be built based on the complete code generated by the LLM.

(D) Comparison of whole code and function only. "Accu 1" represents the generation accuracy in the first stage, while "Accu 2" denotes the accuracy in the second stage. The same applies to "Time 1". The data points represent the scores from five experiments, while the bars represent the average scores. The bar chart for some data uses "Success" and "Fail" as labels, where the former means that 10 rounds of iterative search can be completed within the specified time, while the latter cannot. The error bars represent standard error of the mean across five independent tests. For details on the scoring rules, see Note S8.

Reflection-evolution strategy

The reflection-evolution strategy is a two-stage comprehensive search method, as illustrated in Figure 1. Upon receiving a task, the LLM iteratively generates solution algorithms with executable codes through a self-reflection process.[24] These algorithms are evaluated and sequentially improved based on predefined criteria, typically optimizing the loss for data-driven tasks, resulting in a diverse set of solution methods. To address coding errors introduced by the LLM, an error check-and-correct agent is integrated into the framework to automatically resolve any issues (see "check agent" under Methods for the detailed design).

In the second stage, the algorithms generated in the first stage are refined further. One or more methods may proceed to this stage for additional enhancement. Using a function pool, this stage applies an islands evolution strategy[14,25,26] to evolve the algorithms (Figure 1B). Each evolution round involves generating samples and updating the function pool. Simultaneously, the prompt guiding the LLM's algorithm evolution is refined by merging the best and worst function to improve the search for enhanced solutions. After completing a set number of evolution rounds, the best-performing function from the pool is selected as the final output.

LLM selection

In this study, we utilized GPT-3.5-turbo-0125,[27] a leading LLM developed by OpenAI, as the agent for both stages of our framework without any additional fine-tuning.[28] Our experiments indicate that the overall effectiveness of the reflection-evolution strategy is not significantly influenced by the specific version of the LLM, suggesting that our framework could be adapted to other LLMs accessible via application programming interface (API). However, the outcomes of individual iterative steps may vary depending on the semantic understanding and generation capabilities of the LLM used. For a detailed comparison with other LLMs, including GPT-4o,[29] please refer to Note S3.

Generation patterns

The LLM's search space is determined by its generation pattern. We designed two patterns: the whole pattern and the function-only pattern, illustrated in Figures 2A and 2B, respectively. The whole pattern involves the LLM generating all code components, including raw data acquisition, data preprocessing, and other relevant steps. In contrast, the function-only pattern restricts the LLM to generating only key functions at crucial stages. Our testing found the function-only pattern to be more effective.

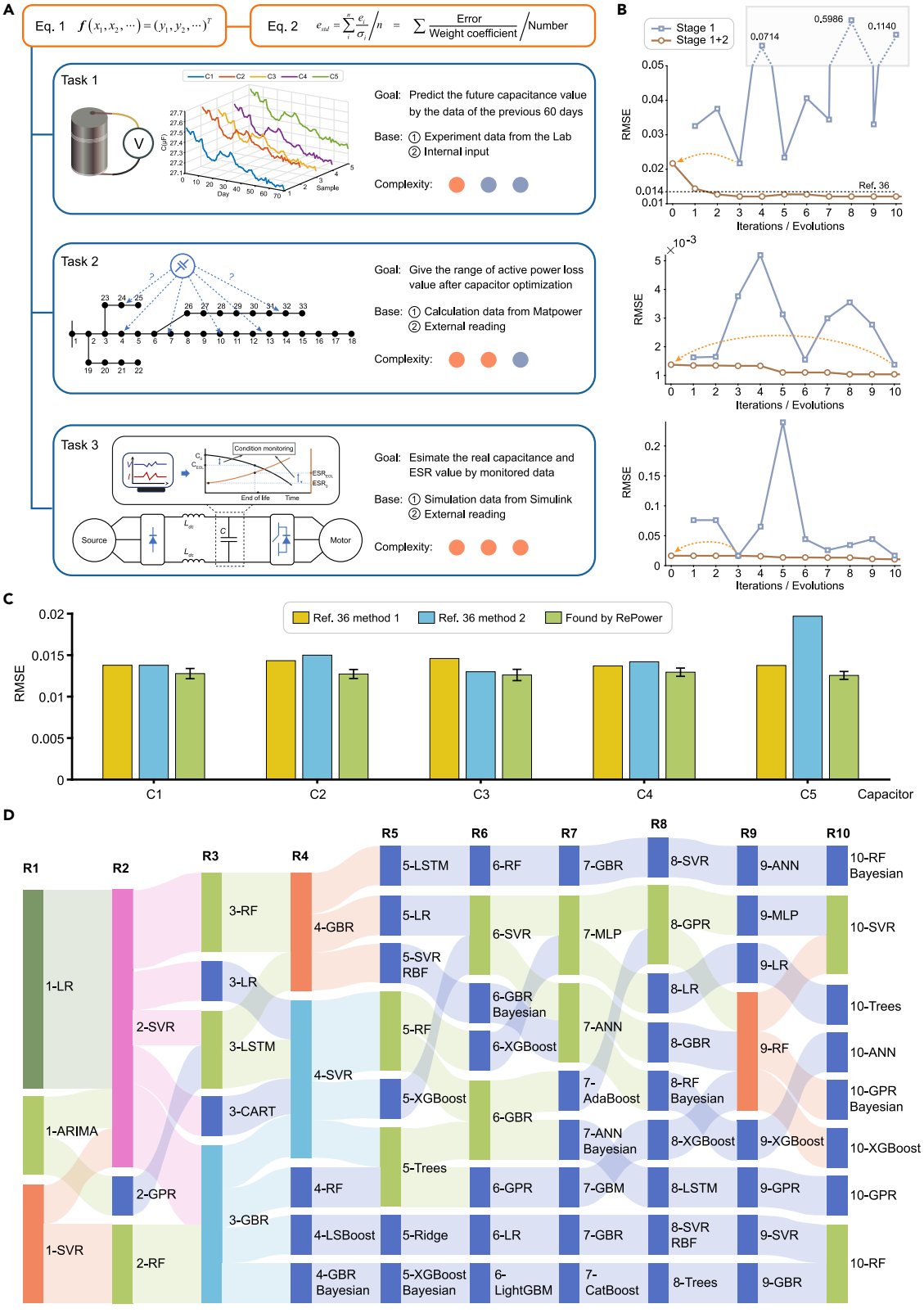
We identified two types of tasks: those with existing solutions and those without. For tasks with existing solutions, researchers can provide a code framework to guide the LLM's generation process. For tasks without existing solutions, the LLM initially generates a complete code framework based on the natural language description of the problem. Key functions are then extracted and generated separately to avoid redundant code. For example, the generation framework shown in Figure 2C is derived from Figure 2A, where essential functions like the "for" loop for prediction and root-mean-square error (RMSE) calculation are abstracted out, forming the streamlined function structure in Figure 2C, task prompt.

Prompt design

The RePower framework uses distinct prompts across its two stages. In the first stage, two prompts are employed: the task prompt and the self-reflection prompt. The task prompt combines the task description with the code framework (Figure 2C), guiding the LLM in generating initial solutions. The self-reflection prompt incorporates the evaluation of the LLM's output and feedback on the next action[30] without including specific details about the function itself.

In the second stage, the prompt is derived from the function pool. This process involves selecting two functions from a randomly chosen group, typically the best- and worst-performing functions, along with their evaluation metrics, such as

CellPress
OPEN ACCESS



(legend on next page)

prediction errors. These components form the function combination prompt (Figure 2C). Both the self-reflection and function combination prompts act as feedback from execution, with distinct prompt generation rules tailored to each stage of the framework.[31]

Interaction platform

Our preset interaction platform consists of an impedance analyzer (Zurich Instruments MFIA [medium frequency impedance analyzer]),[32] a sample, and a computer (Figure 1A). We use the LabOne API,[33] a cross-platform tool that enables data acquisition, hardware control, and signal processing, to facilitate communication between RePower and the instruments. Once the communication interface is established, other devices can also be controlled using similar configurations, as shown in Figure 1C. These components form a multi-measurement platform that allows RePower to autonomously conduct corresponding tests in the laboratory. For detailed design and operation of the physical setup, refer to "physical operation" under Methods.

Power system research tasks

Many power system research tasks can be abstracted into a mathematical problem (Figure 3A); for example, load forecasting uses historical values to predict future values, satisfying this formula.[34] These tasks involve finding a mapping function that links given inputs to desired outputs, with the primary challenge being to determine this function. While the specific inputs and outputs vary (Table S1), influencing the complexity of each task, the underlying structure remains consistent.

We applied RePower to three typical data-driven tasks in power systems, identified through keyword frequency analysis of a large body of literature (Figure S11). The tasks include parameter prediction, power optimization, and state estimation. The final results of these tasks are presented in Figure S2. Additional results from other tasks can be found in Figure S12.

Parameter prediction

Parameter prediction is a common challenge in power systems, such as load forecasting. In this study, we focus on predicting capacitance values as a representative task that integrates both software and hardware aspects. Accurate capacitance prediction is crucial for aging monitoring and fault detection, as a 5% decrease in capacitance indicates the end of a capacitor's lifespan.[35] We use aging data from a prior study, in which five capacitors were monitored for over 70 days using the ZCtek ZC2811D LCR (inductance capacitance resistance) meter.[36] The capacitor is a biaxially oriented polypropylene capacitor modified with high-dielectric-constant nanoparticles with an initial capacitance close to 27.6 μF. It was measured after being placed in an environment of 22°C and 30% relative humidity for a fixed period of time. The aging curves of these five capacitors are shown in Figure 3A, task 1. The benchmark task is a point prediction, which requires predicting the capacitance values for the next 10 days based on data collected over the first 60 days, and the effectiveness of the prediction method is evaluated by calculating the RMSE for the last 10 days. The training set and test set are autonomously partitioned by the LLM. In subsequent tests, the LLM will mostly adopt an 80/20 split, meaning the first four capacitors are used as the training set, and the last capacitor is used as the test set.

Power optimization

Optimization problems are essential across various stages of power transmission and distribution and are critical for power planning designers. We selected a reactive power compensation optimization problem for RePower to handle[37] As we focus on data-driven tasks, the required data are pre-calculated using the particle swarm optimization algorithm and MATPOWER (Matlab-based power system simulation package).[38] The goal is to determine an optimal indicator, such as the maximum active power loss, given a set of optimization parameters. In this optimization problem, it is necessary to reasonably perform capacitor shunt and series compensation in a 33-node system to minimize active power loss. The objective is to minimize active power loss with a total of 9 variables including the initial series compensation point location, the length of the series compensation point's effect, the series compensation resistance value, the initial shunt compensation point location 1, the length of the shunt compensation point's effect 1, the shunt compensation point capacity size 1, the initial shunt compensation point location 2, the length of the shunt compensation point's effect 2, and the shunt compensation point capacity size 2. The constraints include the power flow equation constraints, the maximum series compensation resistance limit, and the maximum shunt compensation capacity limit. The calculation of active power loss is the sum of all branch losses in the 33-node system. Finally, the maximum and minimum values of active power loss need to be calculated, and the evaluation of the method's effectiveness is the RMSE of active power loss.

State estimation

State estimation is a foundational task in electrical engineering.[39] In this study, we focus on estimating the capacitance and equivalent series resistance (ESR) of the DC (direct current)-link capacitor[40] in power electronic converters, a prerequisite task for capacitance prediction.[41,42] Using Simulink, we simulate the operating voltage and current of the DC-link capacitor under various conditions (Figure S5). The task aims to estimate the capacitance value and ESR of the DC-link capacitor using voltage and current data obtained from simulation over 101 time units, where the effectiveness of the method is evaluated by the RMSE of these two parameters. In this context, we model

Figure 3. Task profile and evolution results

(A) Task profile. Three tasks are outlined: the goal (expected results for the research task), base (key steps before starting the system), and complexity (assessment of task difficulty).

(B) Performance of the two stages. The first stage iterates 10 times, selecting the optimal function as the initial value for the second-stage evolutions.

(C) The comparison between the final iteration result of task 1 and the existing solution. The abscissa represents different capacitors, while the ordinate represents the predicted error of this method on this capacitor. The error bars represent standard error of the mean in five tests.

(D) Stage 1 method searching by the LLM. The statistics here are for three tasks described in the main text. The names of these methods were described to users by the LLM, but in the actual code it wrote, the LLM may not have used the corresponding methods, resulting in a certain degree of information bias. For specific methods in abbreviation, please refer to Table S4.

the capacitor using an ideal capacitor in series with a resistor. To simulate the decay process of the capacitor, we set the resistance value to vary between 0.05 and 0.1 Ω and the capacitance value to vary between 2 and 1.6 mF. We continue to employ point prediction, utilizing a total of 202 voltage and current data points to predict the capacitor and ESR values. The training set and test set are still autonomously divided by the LLM.

Before conducting a complete test, we assessed the performance of the three tasks using the two previously described generation patterns: whole generation and function-only generation. As shown in Figure 2D, the function-only generation pattern outperforms whole generation in tasks 1 and 3 but slightly underperforms in task 2. In task 1, which involves direct input of raw data (Figure 3A), generating complete code leads to repeated data copying by the LLM, causing errors and reduced accuracy. In task 3, which requires reading data from external sources, the complexity of data type conversion significantly decreases accuracy during the first stage. As a result, the second-stage accuracy for tasks 1 and 3 is not displayed in Figure 2D because the first stage could not generate the required 10 functions due to the extended logical chain and increased error probability in complete generation. GPT-3.5 struggles to generate all functions within the allowed time and iteration limits. While both the whole and function-only generation patterns can be used with an LLM like GPT-3.5, the function-only pattern generally performs better for tasks involving complex data types or direct data inputs in RePower due to its shorter reasoning steps.

Performance on the three tasks

We evaluated the accuracy of calculations using normalized RMSE for multiple outputs (Figure 3A, weighted average error) and standard RMSE for single outputs. As shown in Figure 3B, the results of the first stage did not improve with increased search iterations, which aligns with our expectations. In the first stage, the objective is not to find the best methods but to generate diverse solutions that can guide the evolutionary process in the second stage. After only 10 rounds of evolution in stage 2, RePower's performance on task 1 surpassed the benchmark method from Zhang et al.[36] (smoothing spine with RMSE ≈ 0.014, which uses a piecewise low-order polynomial fitting method to make the curve smoother while maintaining overall stability and effectively preventing overfitting[43]), which used the same dataset and claimed to be the best. This highlights RePower's capability to enhance and evolve methods through automated processes. The line chart in Figure 3B shows a continuous error reduction with increased iterations in the second stage until stabilization, indicating successful ongoing algorithm evolution under RePower. Although similar trends were observed across tasks, task 1 exhibited a notably faster error decline. After 20 rounds of the two-stage evolutionary process, task 1's RMSE decreased from 0.0144 to 0.0119, representing an average reduction of 9.29% compared to Zhang et al.[36] (Figure 3C). The method evolved from initial linear regression to compound linear regression. For task 2, the RMSE decreased from 0.00137 to 0.000825, with the method evolving from a simple decision tree ensemble to one incorporating Bayesian optimization and feature dimension reduction. Task 3's RMSE dropped from 0.0165 to 0.00971, transitioning from support vector regression (SVR) to SVR with added polynomial features. All tasks achieved favorable results, demonstrating the effectiveness of RePower's automatic solution evolution.

In task 1, the overall trend of capacitance value attenuation is linear but exhibits fluctuations over different periods (Figure 3A). In the process of algorithm evolution, LLMs initially attempt polynomial fitting from low order to high order, followed by a comparison with the SVR method. Subsequently, hyperparameter tuning is applied to SVR to enhance prediction accuracy. The final evolved solution combines linear and SVR approaches, capturing the general linear decline with added nonlinear adjustments for observed disturbances (Figure S2A). Task 2 involves nine optimization variable parameters. During evolution, the LLM employs principal-component analysis (PCA) for dimensionality reduction and feature selection, followed by Bayesian optimization for hyperparameter optimization and regression trees for prediction (Figure S2B). Since the nine-dimensional raw data contain fixed elements that do not influence the final outcome, dimensionality reduction improves performance by removing irrelevant components. Task 3 deals with time-series data of voltage and current, requiring feature extraction to reveal the relationships between state parameters, capacitance values, and ESR. The final solution algorithm developed by RePower first extracts features using a cubic polynomial, which are then fed into a fully connected feedforward neural network for training and prediction. The hyperparameters of the neural network were not adjusted or optimized (Figure S2C).

Comparison with other LLM-based evolutionary frameworks

To further validate RePower's effectiveness, we compared it with two other evolutionary frameworks similar to RePower, LEADE (LLM-enhanced adaptive evolutionary search)[44] and ReEvo (Reflective Evolution).[45] Comparative diagrams of these frameworks and RePower are provided in in Figure 4.

LEADE begins by initializing 30 functions in response to a given task prompt, selecting 10 of these to form the initial function group. From this group, adaptive crossover and mutation operations are applied to generate new functions, which are then merged with the original set, expanding the function group. This completes one cycle of the evolutionary process. ReEvo follows the same initialization process as LEADE but introduces short-term and long-term reflections during the crossover and mutation phases. Unlike LEADE, ReEvo replaces the original function group with the newly generated one rather than merging the two. In contrast, RePower employs an initial reflection phase to explore diverse methods, followed by a group evolutionary search to iteratively optimize these methods, functioning as a collaborative evolutionary system.

The performance of the three frameworks across the first three tasks is presented in Figure 5. It reveals that LEADE outperformed RePower only in task 1, while RePower identified superior methods for the remaining two tasks. This disparity arises because task 1 is relatively simple, involving only a single changing parameter, which makes heuristic search strategies effective in finding better predictive methods. However, this approach becomes less effective in more complex problems, as reflected in tasks 2 and 3, where LEADE and ReEvo exhibit plateaus, indicating that iterative searches reached stagnation.

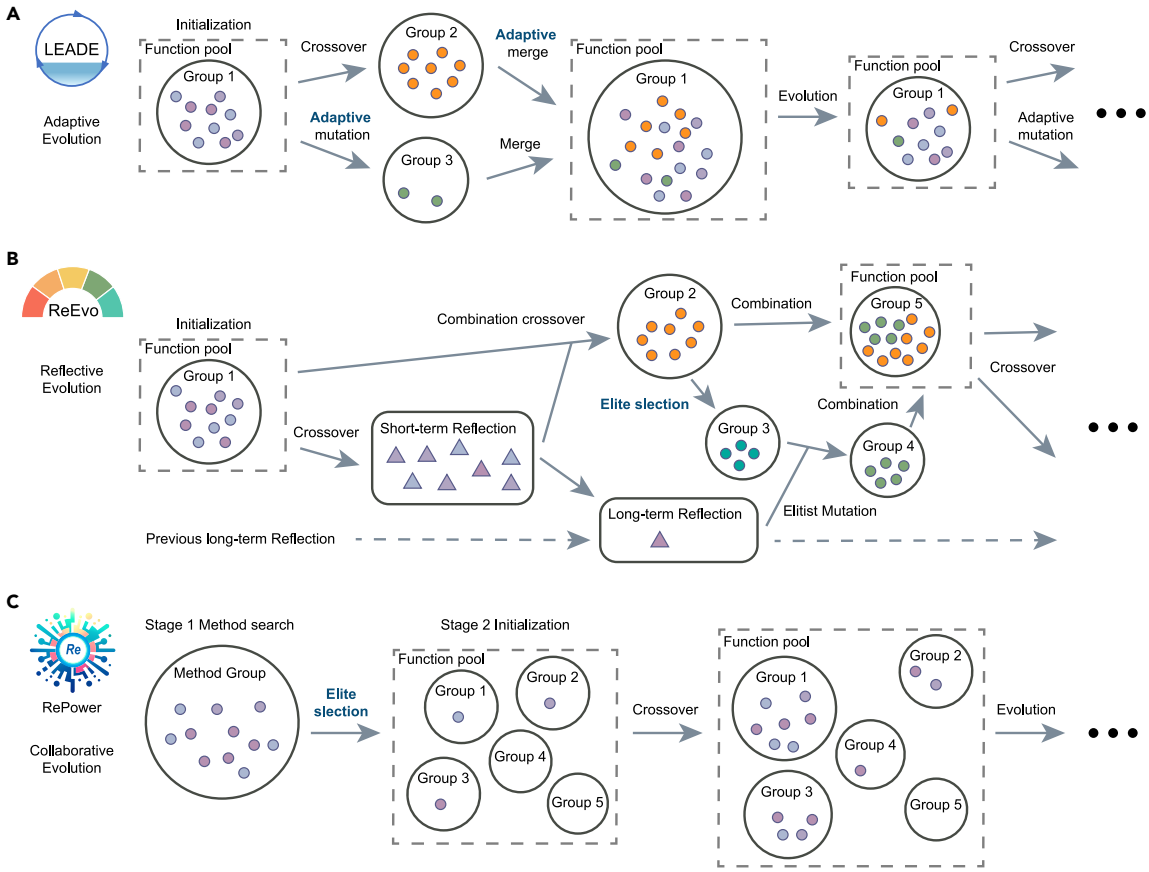


Figure 4. Workflow diagram comparison of different evolutionary frameworks

(A) Adaptive evolution of LEADE. It is characterized by adaptively performing crossover and mutation processes according to the fitness of the function.

(B) Reflective evolution (ReEvo). Its characteristic is to add long-term and short-term reflection of speech in the process of crossover and mutation, and this reflection is also a kind of crossover.

(C) Collaborative evolution of RePower. Its characteristic is the collaboration of a two-stage search process, where the first stage involves method search, and the second stage entails group evolutionary search.

In many cases, LEADE and ReEvo tend to converge on linear or polynomial regression, making incremental improvements without evolving to more sophisticated methods, contributing to their underperformance in challenging tasks. It is noteworthy that RePower generally requires a longer search time owing to its more complex reflection and evolution strategy. Yet, it is worthwhile to trade long thinking time for the quality of the solution.

Performance factors and explorations of RePower

Given RePower's numerous operational steps, we focused on factors affecting its core evolutionary performance. Figure 6 presents the impact of several key explorations. Our results show that both stages of RePower's self-improvement process are critical and cannot be substituted for one another. The first stage alone does not achieve continuous improvement (Figure 3B), while the second stage alone, though offering partial evolution (Figure 6A), performs worse in prediction error (53.40% vs. 38.27%) and has a higher generation error rate compared to the combined two-stage approach (Figure 6B). Generation error rate is defined as the proportion of incorrectly executed samples out of the total generated samples (see Note S7), demonstrating that RePower outperforms FunSearch's single-stage evolution. Without the first stage, there is insufficient diversity in the initial functions, often causing the LLM to default to linear regression or occasionally other methods, like random forests (Figure 6C). This lack of diversity limits the LLM's ability to direct the evolution process, making subsequent searches more random and reducing overall performance. In the absence of robust initial function screening, the average generation error rate increases, resulting in suboptimal final outcomes. Given the significant influence of initial values from stage 1 on stage 2's performance, we further examined how varying initial values affects stage 2's output (Figure 6D). Different starting values produced different results, with the best function from stage 1 not always leading to the optimal outcome due to the risk of getting trapped in local optima. To mitigate this limitation, we introduced multiple initial functions (denoted as k) from stage 1 into stage 2 for iteration. The results indicated that the best performance was achieved with $k = 3$ (Figure 6E). A smaller k reduces search diversity, while a larger k dilutes the optimal function by introducing many non-beneficial functions, potentially compromising overall performance.

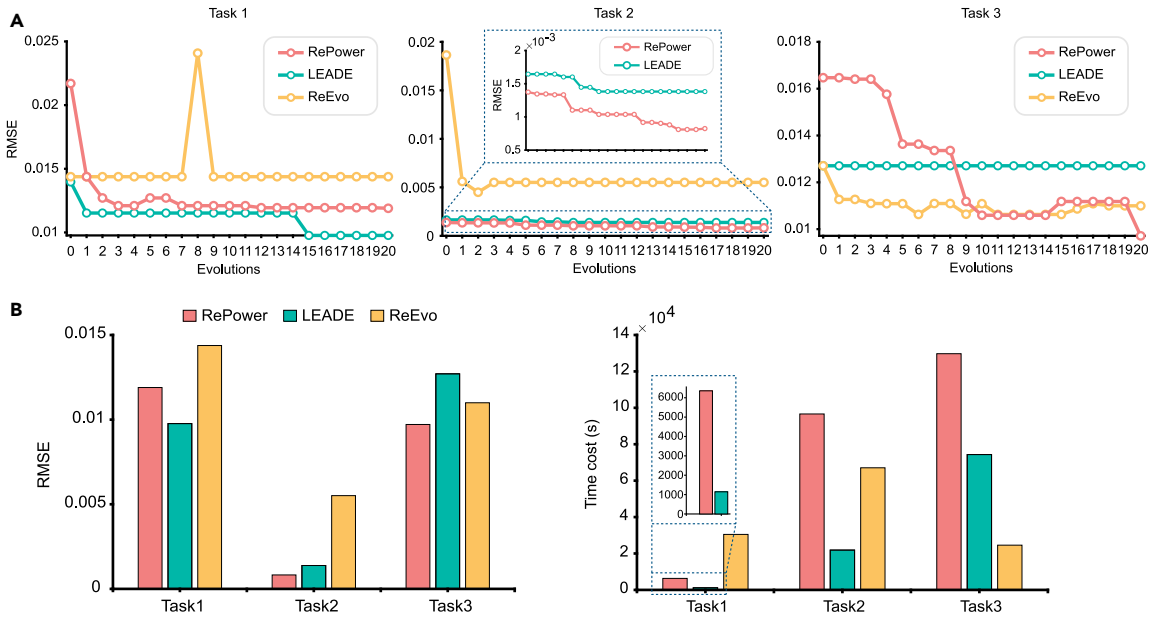


Figure 5. Task results comparison using different evolutionary frameworks

(A) The evolutionary process comparison of the three frameworks. The result of RePower is the second-stage evolutionary process that connects the first stage.

(B) The final result comparison of the three frameworks.

DISCUSSION

We demonstrated RePower's effectiveness as a research designer capable of conceptualizing new algorithms and continuously refining them. Its strong adaptability to physical interfaces (Figure 1C) makes RePower highly suitable for client-side deployment. Compared to other evolutionary frameworks, RePower's approach of evolving multiple function groups enhances individual diversity, optimizing overall performance (Figure 5). The entire process is highly automated, with task completion times significantly shorter than human efforts—less than half the time—while maintaining reasonable costs (Figure S15). RePower's efficiency and automation highlight its potential to revolutionize research processes in power systems and beyond.

Despite its strengths, RePower has some limitations. Due to constraints in laboratory equipment, interactions with more complex devices for enhanced designs remain incomplete but offer opportunities for future improvement. We also aim to develop a unified generation framework compatible with the LLM environment, designed to handle diverse data operations to produce outputs. The current framework, primarily focused on predictive functions, may be inadequate, as it requires identifying commonalities across various methods. Additionally, while testing large models typically involves extensive datasets with explicit answers,[46] our evaluation relied on a limited number of smaller cases with unknown or open-ended outcomes. Despite these limitations, RePower demonstrates significant potential in designing and evolving solutions, highlighting the need for further exploration and refinement.

RePower's reflection-evolution strategy effectively broadens the search space while efficiently completing tasks. Simply generating numerous function samples and screening them in the second stage, without method selection in the first stage, would not achieve optimal results. This is because an LLM without fine-tuning or contextual information tends to generate algorithms based on the most probable outcomes from its pre-training, often defaulting to basic methods like linear regression, thereby missing opportunities to explore diverse approaches. Similarly, without the in-depth reflection provided in the second stage, many methods remain surface level and fail to reach their full potential. RePower's success lies in its reflection-evolution strategy, which integrates horizontal and vertical search, combines long-term and short-term memory, and incorporates human-like feedback with execution. This comprehensive approach enables RePower to navigate complex tasks with depth and adaptability. Looking ahead, we anticipate further improvements in this automated system, enhancing its role as a versatile research assistant across various fields. RePower holds the promise of advancing scientific exploration, providing novel insights, and supporting human researchers in unprecedented ways.

METHODS

Datasets

This study did not use any open-source datasets in the main text; instead, we independently constructed datasets tailored to each specific task. The capacitance data were directly measured in our laboratory, as detailed in Zhang et al.[36] Data for the other two tasks were generated using MATLAB simulations (Figure S5). For additional tasks presented in the supplemental information, we employed datasets from the UCI (University of California Irvine) Machine Learning Repository.[47]

LLM

The LLM serves as the core of RePower's innovation, with GPT-3.5-turbo-0125 used for generating task-specific instructions

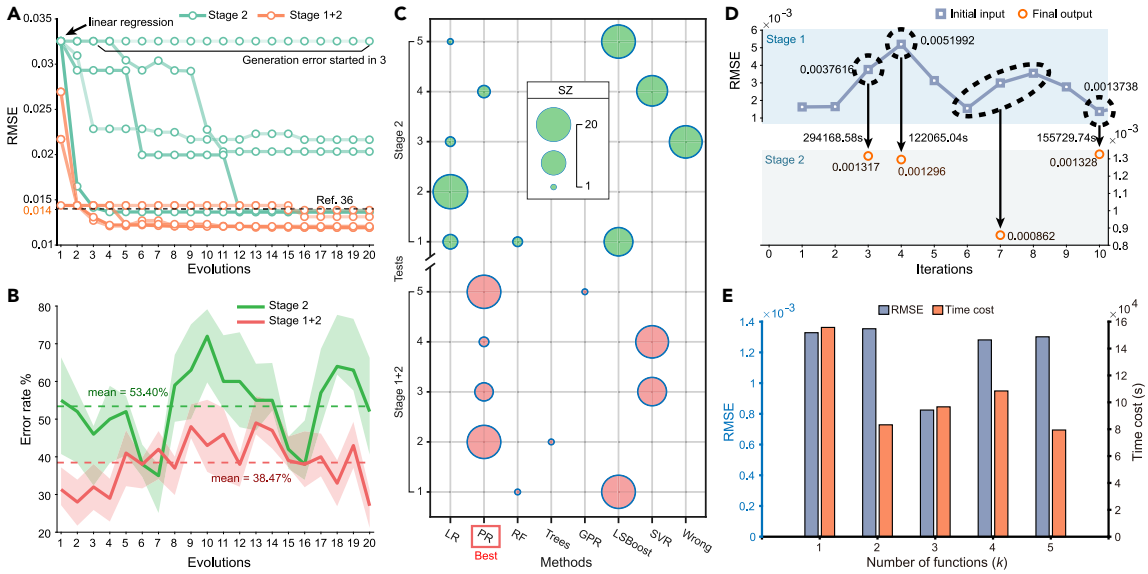


Figure 6. Exploration of the RePower's performance factors

(A) RMSE comparison. This compares RMSE of using only stage 2 versus both stage 1 and 2 of task 1. RMSE represents the normalized standard error when there are multiple outputs.

(B) Average error rate comparison. This compares the average error rate between using only stage 2 and both stages 1 and 2 of task 1. The width of the band represents the standard error of the mean.

(C) Method comparison. Here, we compare the methods that emerged during 20 evolutionary processes of task 1, using only stage 2 and both stage 1 and 2, respectively, across 5 experiments. SZ represents size, indicating that the frequency is presented in the form of size. The "PR" method is the best approach suitable for this task (found by our system), whereas the "Wrong" method implies that there is a flaw in the computational logic.

(D) Initial value selection performance. This compares the performance of initial value selection in the first stage of task 2. Results are based on selecting functions enclosed by black dashed circles to enter the second stage of iteration.

(E) Function number comparison. This compares the performance of different quantities of functions transitioning from stage 1 to stage 2 of task 2.

and algorithm codes. A comparative analysis between GPT-3.5 and the more advanced GPT-4o[29] revealed only marginal improvements in individual results with GPT-4o without a significant impact on overall self-improvement performance. Both models also demonstrated similar error types and frequencies (Figure S10). Based on these findings, we selected GPT-3.5 as the primary tool for our system.

Prompt generation

In stage 1, apart from the manually provided task prompts, each self-reflection prompt was automatically generated based on preset scenarios involving simple conditional statements.[48] If execution was correct, then the response was constructed using the return value, typically highlighting a prediction error and suggesting alternative methods for subsequent steps. In cases of errors or timeouts, the prompt included the error code and message to attempt a fix, with the check agent deployed to address these issues. If the error could not be resolved (e.g., the LLM failed to understand the error and made arbitrary changes), the LLM was instructed to regenerate a new solution to ensure the iteration continued without interruption.

Stage 2 prompts were generated using a combination of the best and worst functions from a randomly selected group within the function pool (Figure 2C, function combination prompt). This approach, inspired by evolutionary algorithms, was designed to help the LLM identify functional differences and drive continuous improvement. Experimental results demonstrated that this combination was more effective in finding breakthroughs (Figure S3A). Combining the best and worst functions enhanced evolutionary outcomes by maximizing mutation potential and avoiding local optima. Throughout the process, we prioritized group optimality over individual performance, allowing for fluctuations that ensured significant deviations in the evolutionary path and enhanced the overall refinement process.

Check agent

In the first stage of iteration, our goal was for the LLM to maintain contextual memory to correctly generate various methods for sequential execution. However, the LLM-generated code frequently contained errors that could disrupt the program. To address this, we developed an LLM-based check agent to identify and correct code errors (Figure S1). This agent operates as a proxy for the main chat, focusing specifically on error correction. When an error occurs in the main chat, the error code and message are transferred to the check agent, which guides the LLM in repairing the issue. Once corrected, the revised code and updated return value are sent back to the main chat. If the errors cannot be resolved despite multiple attempts, the main chat's context is reset, and the process is regenerated to ensure continuous iteration.

Evolution strategy

The evolution strategy is central to the second stage of RePower's iterative process. The first stage involves a horizontal

Algorithm 1. Evolution strategy

```
Initialize individuals, groups: f, g = { f }
Initialize function pool P = { g }
while i < max evolutions do
  while j < max samples do
    Randomly select g' in P
    Construct prompt s using f in g'
    Generate individuals f' using prompt s
    if f' pass
      Append f' to P
    end if
    Increment j
  end while
  Evolve P
  Increment i
end while
return
```

search conducted by the LLM using a self-reflection strategy. Although these operations could be performed using existing interfaces like MatGPT[49] and Assistant API,[50] we opted to use our own interfaces to facilitate system processing, feedback integration, and strategy improvement. In stage 2, we employed an island-based genetic algorithm for evolutionary search.[51–53] This strategy comprises three hierarchical components: the function pool, groups, and individuals.

(1) The function pool is the largest environment, akin to a society, where all correct output samples generated by the LLM reside as individuals. It serves as the primary space for evolution. (2) Groups are smaller subsets within the function pool, containing function samples that directly influence evolution. When a correct sample is added to the function pool, it is assigned to a group with similar properties, such as average error. Each group marks its best and worst individuals for future prompt construction. Adding new individuals updates the group's performance metrics, including the average error. (3) Individuals are the smallest units in the function pool, each represented by MATLAB code, output information (such as prediction error), and other details (e.g., code length). The terms "individual" and "function" are used interchangeably due to minimal differences between them.

The evolution process unfolds in several steps (Figure 1B). Initially, the function pool is seeded with the best-performing functions from the first stage, which are then sorted into groups. Evolution begins once sufficient samples are generated in each round. Functions selected for evolution are the top k from the first stage, and $k+n$ function groups are formed, where n accounts for additional groups that maintain diversity and store outlier functions during early evolution (Figure 1B shows the effect with $n = 1$). Subsequent experiments identified optimal performance with $k = 3$ and $n = 2$ (Figure S3B). During sample generation, correctly generated functions are added to the group with the closest error value. After all samples are generated, the function pool undergoes evolution. Functions are randomly selected from the top-performing groups to replace those in other groups, updating group errors and identifying the best and worst individuals.

To prevent degradation due to an excess of individuals in a single group (where inferior functions could outnumber and drive out superior ones),[52,54] any group exceeding 10 individuals randomly retains 9 plus the current optimal function. This approach ensures that each group maintains 10 individuals, preserving diversity and stability in the evolutionary process.

Physical operation

The physical operations in this study are confined to the direct control of electronic test equipment for data acquisition, excluding any mechanical motion components. As shown in Figure 1C, we developed a multi-measurement platform capable of controlling three devices: the MFIA impedance analyzer,[32] the Alpha-A broadband dielectric spectrometer,[55] and the Keithley system sourcemeter.[56] The MFIA impedance analyzer connects directly to the computer via USB (universal serial bus) (A-B), while the Alpha-A spectrometer and the Keithley sourcemeter communicate through the National Instruments GPIB (general purpose interface bus)-USB-HS (high speed) interface.[57] To ensure safety, particularly in light of potential LLM hallucinations,[12,58] the LLM generates only control instructions, which are then executed by pre-written API programs. This approach simplifies the process and ensures safety. For example, when using the impedance analyzer, RePower generates control instructions that are sent to the device via its API[33] to conduct the test. The device then returns the data for the specified parameters, which is subsequently used in RePower's two-stage iterative self-improvement process to further refine the solution for the specific task (Figures S7–S9).

Extended skeleton

To improve the efficiency of the LLM, we developed an extended skeleton framework that abstracts repetitive steps commonly found in our method, such as data extraction and error calculation. This framework focuses the LLM on generating only the key steps of each process, minimizing redundant operations. By abstracting these common steps, the framework enables the LLM to employ heuristic thinking to identify better alternative approaches and incrementally refine solutions. Additionally, this framework helps prevent the LLM from generating solutions that, while producing correct results, do not align with the task's specific requirements. An example of a simplified framework is shown in Figure 2C, illustrating how structured guidance enhances logical coherence in the LLM's output.

RESOURCE AVAILABILITY

Lead contact

Requests for further information, resources, and reagents should be directed to and will be fulfilled by the lead contact, Zhi-Min Dang (dangzm@tsinghua.edu.cn).

Materials availability

This study did not generate new materials or reagents.

Data and code availability

- All original code with experiment data has been deposited at Zenodo[59] (https://doi.org/10.5281/zenodo.14216501) and is publicly available as of the date of publication.
- ChatGPT conversation generation examples of the 6 tasks mentioned in this study have been deposited at Zenodo[60] at https://doi.org/10.5281/zenodo.14219850 and are publicly available as of the date of publication.

- Any additional information required to reanalyze the data reported in this paper is available from the lead contact upon request.

ACKNOWLEDGMENTS

This work was financially supported by National Key Research and Development Program of China (2021YFB2401504)

AUTHOR CONTRIBUTIONS

Conceptualization, Z.M.D., Y.-X.L., and M.J.; methodology, M.J.; investigation, Y.-X.L., Y.-X.Z., and M.J.; visualization, Y.-X.L. and Y.-X.Z.; writing – original draft, Y.-X.L.; writing – review & editing, J.W., G.H., and Z.M.D.; funding acquisition, S.-L.Z. and Z.M.D.; project administration, S.L.Z.; supervision, Z.M.D.

DECLARATION OF INTERESTS

The authors declare no competing interests.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES

During the preparation of this work the authors used GPT-4o in order to polish the academic writing. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

SUPPLEMENTAL INFORMATION

Supplemental information can be found online at https://doi.org/10.1016/j.patter.2025.101211.

Received: November 23, 2024
Revised: January 27, 2025
Accepted: March 6, 2025
Published: March 31, 2025

REFERENCES

1. Mei, S., Zhang, X., and Cao, M. (2011). Power Grid Complexity (Tsinghua University Press, Beijing and Springer-Verlag Berlin Heidelberg).
2. Lv, C., Liu, J., and Zhang, Y. (2022). A data-driven method for anomaly detection and aging model parameter estimation of capacitors based on condition monitoring. Microelectron. Reliab. *138*, 114646. https://doi.org/10.1016/j.microrel.2022.114646.
3. Liu, Z., Fang, L., Jiang, D., and Qu, R. (2022). A Machine-Learning-Based Fault Diagnosis Method With Adaptive Secondary Sampling for Multiphase Drive Systems. IEEE Trans. Power Electron. *37*, 8767–8772. https://doi.org/10.1109/tpel.2022.3153797.
4. Wu, W., Qiu, L., Liu, X., Guo, F., Rodriguez, J., Ma, J., and Fang, Y. (2022). Data-Driven Iterative Learning Predictive Control for Power Converters. IEEE Trans. Power Electron. *37*, 14028–14033. https://doi.org/10.1109/tpel.2022.3194518.
5. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language Models are Few-Shot Learners. Proceedings of the 34th International Conference on Neural Information Processing Systems (NeuIPS). In Curran Associates Inc., H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, eds., pp. 1877–1901.
6. Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. (2022). Emergent Abilities of Large Language Models. Preprint at arXiv. https://doi.org/10.48550/arXiv.2206.07682.
7. Dai, D., Sun, Y., Dong, L., Hao, Y., Sui, Z., and Wei, F. (2022). Why Can GPT Learn In-Context? Language Models Secretly Perform Gradient Descent as Meta-Optimizers. Preprint at arXiv. https://doi.org/10.48550/arXiv.2212.10559.
8. Schäfer, M., Nadi, S., Eghbali, A., and Tip, F. (2023). An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. IEEE Trans. Softw. Eng. *50*, 1–21. https://doi.org/10.1109/tse.2023.3334955.
9. Kwon, S., Lee, S., Kim, T., Ryu, D., and Baik, J. (2023). Exploring LLM-based Automated Repairing of Ansible Script in Edge-Cloud Infrastructures. J. Web Eng. *22*, 889–912. https://doi.org/10.13052/jwe1540-9589.2263.
10. Singhal, K., Azizi, S., Tu, T., Mahdavi, S.S., Wei, J., Chung, H.W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., et al. (2023). Large language models encode clinical knowledge. Nature *620*, 172–180. https://doi.org/10.1038/s41586-023-06291-2.
11. Wang, Z., Chen, A., Tao, K., Han, Y., and Li, J. (2024). MatGPT: A Vane of Materials Informatics from Past, Present, to Future. Adv. Mater. *36*, e2306733. https://doi.org/10.1002/adma.202306733.
12. Boiko, D.A., MacKnight, R., Kline, B., and Gomes, G. (2023). Autonomous chemical research with large language models. Nature *624*, 570–578. https://doi.org/10.1038/s41586-023-06792-0.
13. M Bran, A., Cox, S., Schilter, O., Baldassari, C., White, A.D., and Schwaller, P. (2024). Augmenting large language models with chemistry tools. Nat. Mach. Intell. *6*, 525–535. https://doi.org/10.1038/s42256-024-00832-8.
14. Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., et al. (2024). Mathematical discoveries from program search with large language models. Nature *625*, 468–475. https://doi.org/10.1038/s41586-023-06924-6.
15. Huang, C., Li, S., Liu, R., Wang, H., and Chen, Y. (2023). Large Foundation Models for Power Systems. Preprint at arXiv. https://doi.org/10.48550/arXiv.2312.07044.
16. Majumder, S., Dong, L., Doudi, F., Cai, Y., Tian, C., Kalathil, D., Ding, K., Thatte, A.A., Li, N., and Xie, L. (2024). Exploring the capabilities and limitations of large language models in the electric energy sector. Joule *8*, 1544–1549. https://doi.org/10.1016/j.joule.2024.05.009.
17. Arnautov, K.V., and Akimov, D.A. (2024). In Application of Large Language Models for Optimization of Electric Power System States. 2024 Conference of Young Researchers in Electrical and Electronic Engineering (ElCon)., S. Shaposhnikov, ed. (IEEE), pp. 314–317. https://doi.org/10.1109/elcon61730.2024.10468377.
18. Yan, Z., and Xu, Y. (2024). Real-Time Optimal Power Flow with Linguistic Stipulations: Integrating GPT-Agent and Deep Reinforcement Learning. IEEE Trans. Power Syst. *39*, 4747–4750. https://doi.org/10.1109/tpwrs.2023.3338961.
19. Shahzad, K., Iqbal, S., and Moazam Fraz, M. (2023). Automated Solution Development for Smart Grids: Tapping the Power of Large Language Models. In 2023 17th International Conference on Engineering of Modern Electric Systems (EMES) (IEEE), pp. 1–4. https://doi.org/10.1109/emes58375.2023.10171681.
20. Ruan, J., Liang, G., Zhao, H., Liu, G., Sun, X., Qiu, J., Xu, Z., Wen, F., and Dong, Z.Y. (2024). Applying Large Language Models to Power Systems: Potential Security Threats. IEEE Trans. Smart Grid *15*, 3333–3336. https://doi.org/10.1109/tsg.2024.3373256.
21. Xu, X., Jia, Y., Xu, Y., Xu, Z., Chai, S., and Lai, C.S. (2020). A Multi-Agent Reinforcement Learning-Based Data-Driven Method for Home Energy Management. IEEE Trans. Smart Grid *11*, 3201–3211. https://doi.org/10.1109/tsg.2020.2971427.
22. Yuan, Y., Wang, Y., and Wang, Z. (2024). A Data-Driven Framework for Power System Event Type Identification via Safe Semi-Supervised Techniques. IEEE Trans. Power Syst. *39*, 1460–1471. https://doi.org/10.1109/tpwrs.2023.3266153.
23. Zhang, Y.-X., Feng, Q.-K., Zhong, S.-L., Pei, J.-Y., Chen, F.-Y., He, G.-N., and Dang, Z.-M. (2021). Digital twin accelerating development of

metallized film capacitor: Key issues, framework design and prospects. Energy Rep. *7*, 7704–7715. https://doi.org/10.1016/j.egyr.2021.10.116.

24. Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., et al. (2023). Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback. Preprint at arXiv. https://doi.org/10.48550/arxiv.2302.12813.
25. Tanese, R. (1989). Distributed Genetic Algorithms for Function Optimization (Deep Blue (University of Michigan)).
26. Cantú-Paz, E., and Goldberg, D.E. (1999). On the Scalability of Parallel Genetic Algorithms. Evol. Comput. *7*, 429–449. https://doi.org/10.1162/evco.1999.7.4.429.
27. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS). In Curran Associates Inc., S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, eds., pp. 27730–27744.
28. Chen, B., Yi, F., and Varró, D. (2023). Prompting or Fine-tuning? A Comparative Study of Large Language Models for Taxonomy Construction. In 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C) (IEEE), pp. 588–596. https://doi.org/10.1109/models-c59198.2023.00097.
29. OpenAI (2024). Gpt-4o release. Openai.com. https://openai.com/index/hello-gpt-4o/.
30. Lu, L.-C., Chen, S.-J., Pai, T.-M., Yu, C.-H., Lee, H., and Sun, S.-H. (2024). LLM Discussion: Enhancing the Creativity of Large Language Models via Discussion Framework and Role-Play. Preprint at arXiv. https://doi.org/10.48550/arXiv.2405.06373.
31. Hu, Y., Tang, X., Yang, H., and Zhang, M. (2024). Case-Based or Rule-Based: How Do Transformers Do the Math?. Preprint at arXiv. https://doi.org/10.48550/arXiv.2402.17709.
32. Zurich Instruments AG (2019). MFIA 500 kHz/5 MHz Impedance Analyzer | Zurich Instruments. www.zhinst.com. https://www.zhinst.com/americas/en/products/mfia-impedance-analyzer.
33. Zurich Instruments AG (2023). LabOne API User Manual. docs.zhinst.com. https://docs.zhinst.com/labone_api_user_manual/index.html.
34. Wang, Q., Li, F., Tang, Y., and Xu, Y. (2019). Integrating Model-Driven and Data-Driven Methods for Power System Frequency Stability Assessment and Control. IEEE Trans. Power Syst. *34*, 4557–4568. https://doi.org/10.1109/tpwrs.2019.2919522.
35. Wu, X., Li, K., Tian, R., Yin, H., Yu, T., Cheng, S., and Chen, C. (2024). A Capacitance Estimation Method for DC-Link Capacitors in Railways Based on Pre-Charging Model and Low Sampling Frequency. IEEE Trans. Power Electron. *39*, 1527–1537. https://doi.org/10.1109/tpel.2023.3322699.
36. Zhang, Y.-X., Feng, Q.-K., Chen, F.-Y., Liu, D.-F., Pei, J.-Y., Zhong, S.-L., Yang, Z., and Dang, Z.-M. (2022). Long-term capacitance variation characteristics, law extraction, single and collaborative prediction of film capacitors at room temperature and humidity. Microelectron. Reliab. *139*, 114845. https://doi.org/10.1016/j.microrel.2022.114845.
37. Dai, C., Chen, W., Zhu, Y., and Zhang, X. (2009). Seeker Optimization Algorithm for Optimal Reactive Power Dispatch. IEEE Trans. Power Syst. *24*, 1218–1231. https://doi.org/10.1109/tpwrs.2009.2021226.
38. Zimmerman, R.D., Murillo-Sánchez, C.E., and Thomas, R.J. (2011). MATPOWER: Steady-State Operations, Planning, and Analysis Tools for Power Systems Research and Education. IEEE Trans. Power Syst. *26*, 12–19. https://doi.org/10.1109/TPWRS.2010.2051168.
39. Course, K., and Nair, P.B. (2023). State estimation of a physical system with unknown governing equations. Nature *622*, 261–267. https://doi.org/10.1038/s41586-023-06574-8.
40. Zhao, Z., Davari, P., Lu, W., Wang, H., and Blaabjerg, F. (2021). An Overview of Condition Monitoring Techniques for Capacitors in DC-Link Applications. IEEE Trans. Power Electron. *36*, 3692–3716. https://doi.org/10.1109/tpel.2020.3023469.
41. McGrew, T., Sysoeva, V., Cheng, C.-H., Miller, C., Scofield, J., and Scott, M.J. (2022). Condition Monitoring of DC-Link Capacitors Using Time–Frequency Analysis and Machine Learning Classification of Conducted EMI. IEEE Trans. Power Electron. *37*, 12606–12618. https://doi.org/10.1109/TPEL.2021.3135873.
42. Yao, B., Wei, X., Zhang, Y., Correia, P., Wu, R., Song, S., Trintis, I., Wang, H., and Wang, H. (2024). Accelerated Degradation Testing and Failure Mechanism Analysis of Metallized Film Capacitors for AC Filtering. IEEE Trans. Power Electron. *39*, 6256–6270. https://doi.org/10.1109/tpel.2024.3360373.
43. Gu, C. (2002). Smoothing Spline ANOVA Models (Springer New York, NY). https://doi.org/10.1007/978-1-4757-3683-0.
44. Tian, H., Han, X., Wu, G., Zhou, Y., Li, S., Wei, J., Ye, D., Wang, W., and Zhang, T. (2024). An LLM-enhanced Multi-objective Evolutionary Search for Autonomous Driving Test Scenario Generation. Preprint at arXiv. https://doi.org/10.48550/arXiv.2406.10857.
45. Ye, H., Wang, J., Cao, Z., and Song, G. (2024). ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. Preprint at arXiv. https://doi.org/10.48550/arxiv.2402.01145.
46. Xu, L., Li, A., Zhu, L., Xue, H., Zhu, C., Zhao, K., He, H., Zhang, X., Kang, Q., and Lan, Z. (2023). SuperCLUE: A Comprehensive Chinese Large Language Model Benchmark. Preprint at arXiv. https://doi.org/10.48550/arXiv.2307.15020.
47. UC Irvine (2023). UCI Machine Learning Repository. archive.ics.uci.edu. https://archive.ics.uci.edu.
48. White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., and Schmidt, D.C. (2023). A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. Preprint at arXiv. https://doi.org/10.48550/arXiv.2302.11382.
49. Takeuchi, T. (2024). toshiakit/MatGPT. GitHub. https://github.com/toshiakit/MatGPT.
50. OpenAI (2024). Assistants API. Openai.com. https://platform.openai.com/docs/assistants/overview.
51. Harada, T., and Alba, E. (2021). Parallel Genetic Algorithms: A Useful Survey. ACM Comput. Surv. *53*, 1–39. https://doi.org/10.1145/3400031.
52. Jun-ichi, K., Hara, A., Takahama, T., and Kido, A. (2013). Island-based differential evolution with varying subpopulation size. In 2013 IEEE 6th International Workshop on Computational Intelligence and Applications (IWCIA) (IEEE), pp. 119–124. https://doi.org/10.1109/iwcia.2013.6624798.
53. Li, C., Yang, S. (2008). An Island Based Hybrid Evolutionary Algorithm for Optimization. Proceedings of the 7th International Conference on Simulated Evolution and Learning. In Springer-Verlag., X. Li, M. Kirley, M. Zhang, D. Green, V. Ciesielski, H. Abbass, Z. Michalewicz, T. Hendtlass, K. Deb, K.C. Tan, J. Branke, Y. Shi, eds., pp. 180–189. https://doi.org/10.1007/978-3-540-89694-4_19.
54. Gong, Y., and Fukunaga, A. (2011). Distributed island-model genetic algorithms using heterogeneous parameter settings. In 2011 IEEE Congress of Evolutionary Computation (CEC) (IEEE), pp. 820–827. https://doi.org/10.1109/cec.2011.5949703.
55. Novocontrol Technologies (2023). Alpha-A Analyzer. Novocontrol.de. https://www.novocontrol.de/php/ana_alpha.php.
56. Tektronix (2024). 2600B Series SMU. Tek.com. https://www.tek.com/en/products/keithley/source-measure-units/2600b-series-sourcemeter.
57. National Instruments (2024). GPIB-USB-HS. Ni.com. https://www.ni.com/en-us/shop/model/gpib-usb-hs.html.
58. Zhang, M., Press, O., Merrill, W., Liu, A., and Smith, N.A. (2023). How Language Model Hallucinations Can Snowball. Preprint at arXiv. https://doi.org/10.48550/arXiv.2305.13534.
59. Liu, Y. (2024). RePower source code and experimental data (1.0). Zenodo. https://doi.org/10.5281/zenodo.14216501.
60. Liu, Y. (2024). ChatGPT conversation examples of 6 tasks. Zenodo. https://doi.org/10.5281/zenodo.14219850.